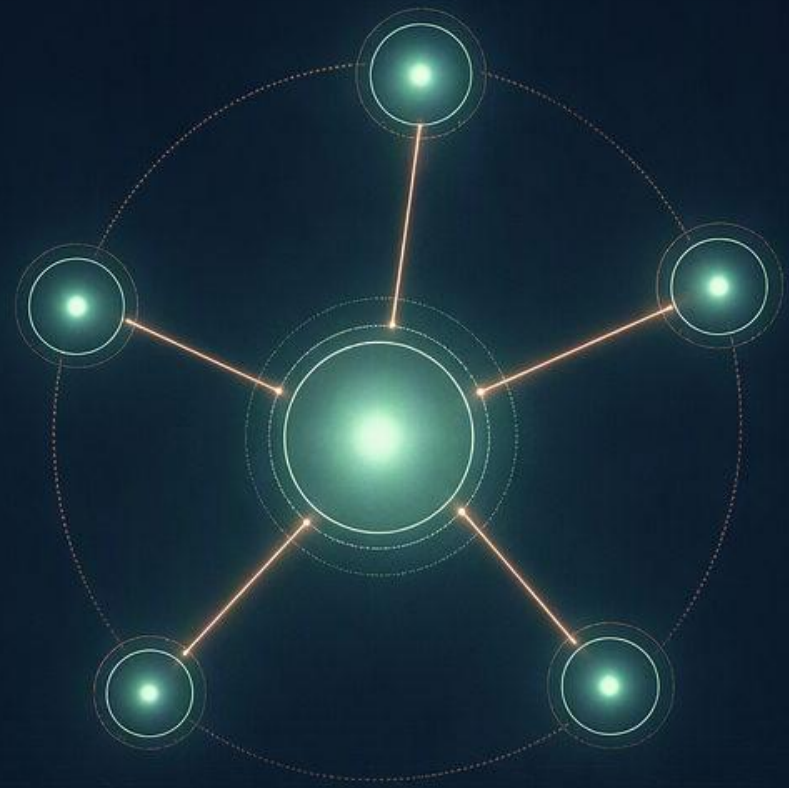
GITHUB COPILOT CLI · PROGRAMOWANIE AGENTOWE

Squad

Zespół agentów AI kierowany przez człowieka

Studium przypadku: Pong Arena



Tomasz Kopacz
Sr Solution Engineer, Microsoft

AGENDA

Dwie części: narzędzie i praktyka

CZĘŚĆ I

Squad — narzędzie

01 Czym jest Squad

02 Założenia i zasady

03 Anatomia folderu .squad/

04 Koordynator i routing

05 Szybki start i codzienna praca

06 Stabilne vs eksperymentalne

CZĘŚĆ II

Studium przypadku: Pong Arena

07 Cel i zakres

08 Zespół, routing, bramki

09 Przebieg rozmowy

10 Ciekawe fragmenty

11 Wynik: działająca aplikacja

12 Wnioski i ograniczenia

CZYM JEST SQUAD

Zespół specjalistów zamiast jednego czatu

„It's not a chatbot wearing hats.”

To nie jeden chatbot przebierany w kolejne role.

„Human-led AI agent teams for any project.” Po opisaniu, co się buduje, Squad proponuje zespół specjalistów (lead, frontend, backend, tester), działający przez GitHub Copilot. Agenci żyją w repozytorium jako pliki, trwają między sesjami, uczą się kodu i współdzielą decyzje.

Narzędzie produktywności dla ludzi, nie zastępstwo inżynierów ani recenzentów. Ludzie odpowiadają za priorytety, akceptacje i finalne zmiany.

v1.0.x
aktualna linia wydań

MIT
licencja open source

~3,3 tys.
gwiazdek na GitHub

Źródło: README repozytorium bradygaster/squad (v1.0.x)

ZAŁOŻENIA

Pięć zasad, na których stoi Squad

01	**Człowiek prowadzi**	Ludzie ustalają priorytety, akceptują zmiany i odpowiadają za finalny kod.
02	**Osobne konteksty**	Każdy agent czyta tylko własną wiedzę i zapisuje to, czego się nauczył — praca jest inspekcjonowalna.
03	**Pamięć w git**	Charter i history.md każdego agenta są w repozytorium: kto sklonuje repo, dostaje zespół z wiedzą.
04	**Równoległość**	Koordynator uruchamia jednocześnie wszystkich agentów, którzy mogą zacząć pracę.
05	**Governance**	Decyzje trafiają do decisions.md, ślady do orchestration-log/ i log/; pamięć jest filtrowana.

Źródło: README bradygaster/squad

ANATOMIA

Folder .squad/ to cały zespół

```
.squad/
├── team.md                roster: kto jest w zespole
├── routing.md             kto co robi, reguły handoffu
├── decisions.md           wspólny mózg: decyzje zespołu
├── ceremonies.md          ceremonie zespołu
├── casting/               policy · registry · history.json — trwałe imiona
├── agents/{name}/         charter.md + history.md
├── agents/scribe/         cichy menedżer pamięci
├── skills/                umiejętności zespołu
├── identity/              now.md · wisdom.md
└── log/                   dziennik pracy; obok orchestration-log/
```

„Commit this folder."
Kto sklonuje repozytorium, dostaje zespół razem z jego wiedzą.

squad upgrade
aktualizuje narzędzie i nigdy nie dotyka stanu .squad/ — pamięć zespołu jest bezpieczna.

Źródło: README bradygaster/squad — sekcja o strukturze .squad/

JAK TO DZIAŁA

Koordynator dzieli, agenci pracują równolegle

Człowiek
prompt, priorytety, akceptacja

→

Koordynator
czyta routing.md, dzieli pracę, pilnuje handoffów i bramek

→

RÓWNOLEGLE

- **Lead**
- **Frontend**
- **Backend**
- **Tester**
- **Scribe — pamięć**

→

ŚLADY W GIT

`decisions.md`
`orchestration-log/`
`log/`
`agents/*/history.md`

Każdy agent dopisuje wnioski do swojego history.md: wiedza kumuluje się między sesjami.

Każdy agent startuje we własnym kontekście; koordynator uruchamia naraz wszystkich, którzy mogą zacząć.

Źródło: README bradygaster/squad

SZYBKI START

Od pustego repozytorium do zespołu

INSTALACJA — JEDNA Z OPCJI

npm (Node ≥ 22.5)

```
npm install -g @bradygaster/squad-cli
```

Windows

```
winget install --id bradygaster.Squad --exact
```

macOS

```
brew install --cask bradygaster/squad/squad
```

VS Code: Copilot Chat → wybrać agenta Squad.

--yolo: Squad wykonuje wiele wywołań narzędzi, więc tryb wymaga zaufania do repozytorium i środowiska.

1 Zainicjować repozytorium

```
git init
```

2 Utworzyć zespół (lub --preset default)

```
squad init
```

3 Zalogować się do GitHub

```
gh auth login
```

4 Uruchomić agenta Squad

```
copilot --agent squad --yolo
```

5 Opisać projekt i zatwierdzić skład („yes”)

„I'm starting a new project. Set up the team. Here's what I'm building: …”

Źródło: README bradygaster/squad — Quick start

CODZIENNA PRACA

Komendy, Ralph, etykiety i @copilot

Komendy CLI

`status · doctor · upgrade · nap`

Także triage, watch, copilot, export/import, externalize, aspire. W czacie: „squad commands”. Samo squad bez argumentów jest przestarzałe.

Ralph — tryb watch

`squad watch --execute --interval 5`

Cykliczny triage issues, uruchamianie agentów, eskalacja do ludzi. Błędy: 4 stopnie eskalacji; stop: plik .squad/ralph-stop.

Etykiety issue

`squad · squad:{member}`

squad — skrzynka do triage przez Lead. squad:{member} — przydział zadania konkretnemu członkowi zespołu.

@copilot i Agentic Workflows

`/squad implement · /squad retro`

squad copilot dołącza coding agenta. Slash-komendy w issue tworzą drafty; przegląd i merge zawsze wykonuje człowiek.

Źródło: README bradygaster/squad — Commands, Ralph, Labels, Agentic Workflows

DOJRZAŁOŚĆ

Co stabilne, a co eksperymentalne

STABILNE

Markdown-first

- Stan zespołu w plikach .squad/, wersjonowany w git
- CLI: init, upgrade, status, doctor, watch
- Copilot CLI oraz VS Code (agent Squad)
- Zalecany do zastosowań produkcyjnych

EKSPERYMENTALNE

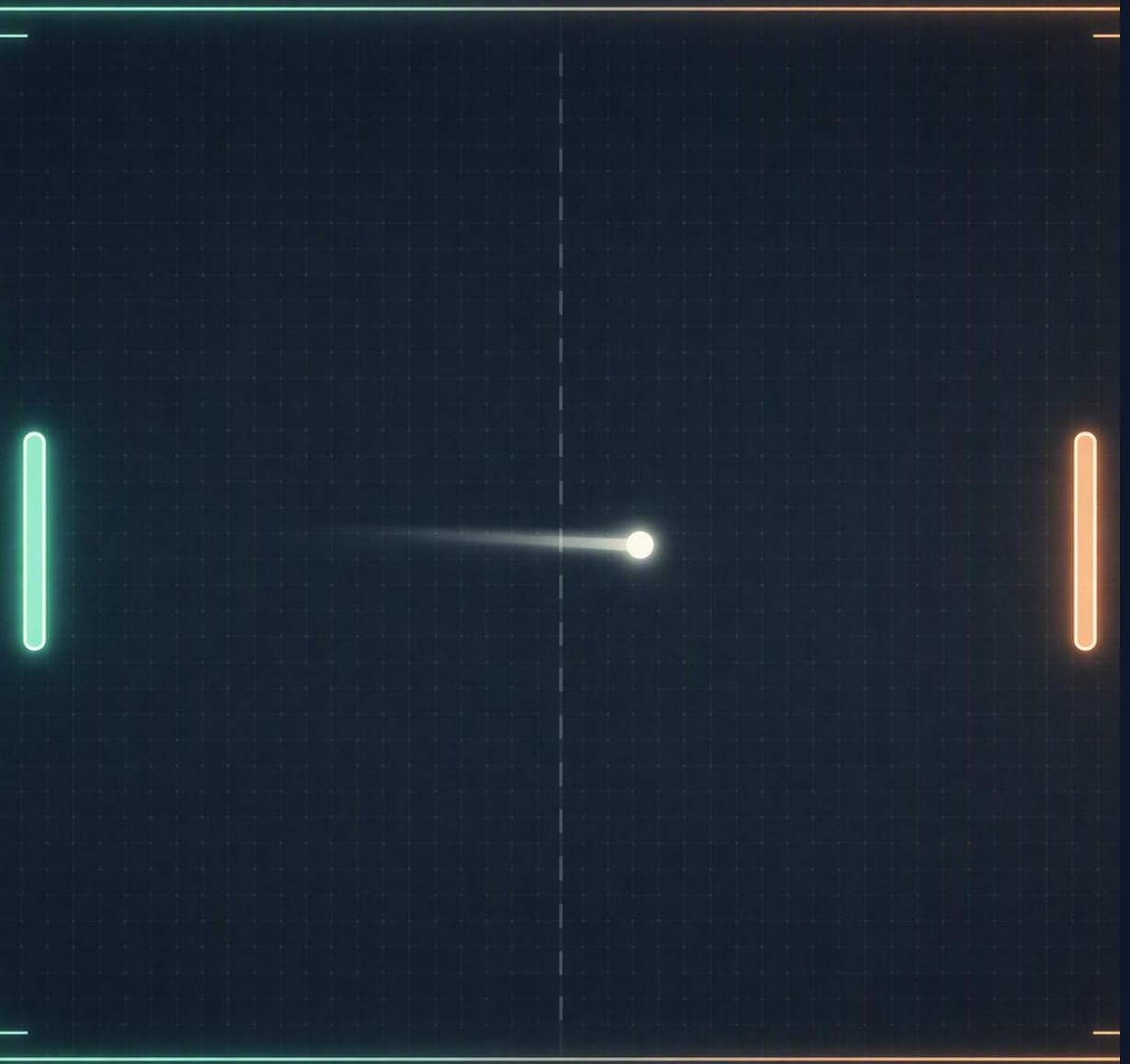
SDK-first mode

```
squad.config.ts · squad build
defineSquad / defineTeam / defineAgent
```

„experimental, known bugs; use markdown-first for production”

Także: pakiet .NET Squad.Agents.AI 1.0.0 oparty na Microsoft Agent Framework.

Źródło: README bradygaster/squad; NuGet Squad.Agents.AI



CZĘŚĆ II · STUDIUM PRZYPADKU

Pong Arena: cel i zakres

`tkopacz/fy27-squad-python01 (prywatne)`
`gałąź squad/pong-arena`

ZAKRES ZATWIERDZONY W TEAM.MD

- Pong w TypeScript, dopracowany wygląd
- Lokalnie dwie osoby: Q/A oraz 8/2, także numpad
- Online dwie przeglądarki przez kod lub link pokoju
- Izolowane, równoległe mecze

ARCHITEKTURA

Node.js + ws, autorytatywny serwer także lokalnie; DOM/Canvas bez frameworka; jedna replika ACA, pokoje w pamięci.

Ciekawostka: repo nazywa się „python01”, a zespół zbudował grę w TypeScript/Node — zakres ustalono w rozmowie.

Źródło: .squad/team.md (repozytorium prywatne)

ZESPÓŁ

Dziesięć ról i coding agent

Człowiek
priorytety · akceptacje · upoważnienia

Koordynator „Squad”
routing · handoffy · bramki recenzji

Skład domyślny (squad init)

- lead
- reviewer
- devrel
- security
- docs

Dodani dla Ponga

- Frontend
- Backend
- Tester
- Infra
- Visual Reviewer

@copilot — coding agent

auto-assign wyłączony

Dobre dopasowanie

bugfixy, testy, lint, zależności, małe funkcje

Wymaga przeglądu

średnie funkcje, refaktoryzacja

Nieodpowiednie

architektura, bezpieczeństwo, niejasne wymagania

Źródło: .squad/team.md

ROUTING I BRAMKI

Najpierw kontrakt, potem równoległość

Backend — najpierw protokół i build
src/shared/protocol.ts

Frontend
równolegle, tylko we własnym zakresie, wg kontraktu

Tester
czeka na działający pion aplikacji

Infra
czeka na build output

„Parallel work stays disjoint”
Frontend nie edytuje manifestów, lockfile ani tsconfig. Zmiana protokołu wymaga lead i handoffu Backend → Frontend / Tester / Infra. „Quick facts → coordinator answers directly.”

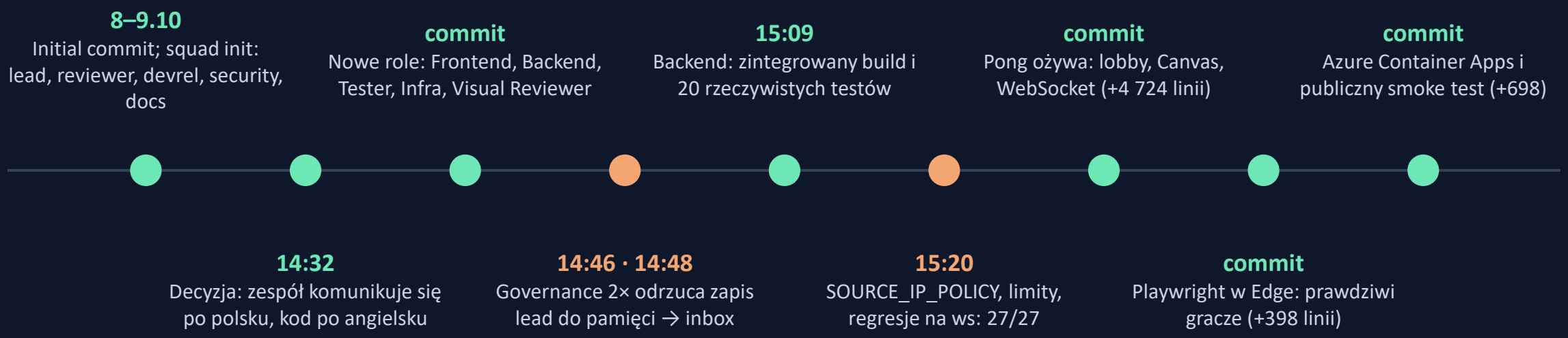
BRAMKI

- ✓ **npm run build**
  `→ dist/…`
- ✓ **npm start**
  `port 8080`
- ✓ **ACA**
  `min/maxReplicas = 1`
- ✓ **Ruch**
  `HTTPS / WSS`
- ✓ **Probe**
  `/healthz`

Źródło: .squad/routing.md

PRZEBIEG ROZMOWY

Dzień pracy zespołu na osi czasu



Godziny CEST, 9.10.2026. Przebieg zrekonstruowany z artefaktów Squad i commitów; pełny zapis czatu nie jest w repozytorium.

CIEKAWOSTKA 1 · 14:46 I 14:48

Governance decyduje, co wolno zapamiętać




```
Rejected as forbidden memory: transient CI/PR status
```

Status CI/PR jest przejściowy, więc nie trafia do trwałej pamięci. Decyzja nie ginie: czeka w decisions/inbox na przegląd. Squad filtruje, co wolno zapamiętać.

Źródło: audit pamięci Squad w repozytorium studium (godz. CEST)

CIEKAWOSTKA 2 · 15:20

Prawdziwe gniazda zamiast mocków

27/27

testów serwera; lint i typecheck PASS

Adres 127.0.0.2 → 127.0.0.1 okazał się niestabilny na Windows niezależnie od aplikacji — odtworzono to na surowym TCP. Agent użył prawdziwego dual-stack IPv4/IPv6, bez skip i bez mocków.

SOURCE_IP_POLICY = direct | shared-proxy
nieznana wartość → fail-fast; X-Forwarded-For / Forwarded ignorowane

8	10	512	128	60
socketów na źródło	create/join na minutę	na min globalnie (shared)	pokoi (shared)	msg/s (shared)

Regresje na ws: 12 socketów i 6 meczów przez wspólny ingress, spoofowane nagłówki ignorowane, limit globalny → 503.

Źródło: notatka agenta Backend, 15:20 CEST

CIEKAWOSTKA 3 · CHARTER

Visual Reviewer ocenia konkretnie

„Nie mylę ozdobności z dopracowaniem. Piłka musi być widoczna, wynik zrozumiały, a błąd pomagać wrócić do gry.”

Motto: „Schludność da się ocenić konkretnie: hierarchia, rytm, kontrast i czytelność w ruchu.”

Nie pisze UI — ocenia i raportuje. Implementację wykonuje Frontend.

KRYTERIA OCENY

1280×720	widok desktop
390×844	widok mobilny
4.5:1	minimalny kontrast

RAPORT

ekran · dowód · problem · priorytet · oczekiwany efekt

Źródło: charter agenta Visual Reviewer (.squad/agents)

CIEKAWOSTKA 4 · DYSCYPLINA RÓL

Każdy robi swoje, nic ponad zakres

„bez rozszerzania scope”

Backend po 15:20

Poinformował Infra, Tester, Visual Reviewer i lead.
Sam nie wykonał deploy ani commit — to nie jego rola.

Upoważnienie do wdrożenia

Tylko grupa zasobów rgfy27-squad-pong.
Region polandcentral.
Dopiero po zielonym build i test.

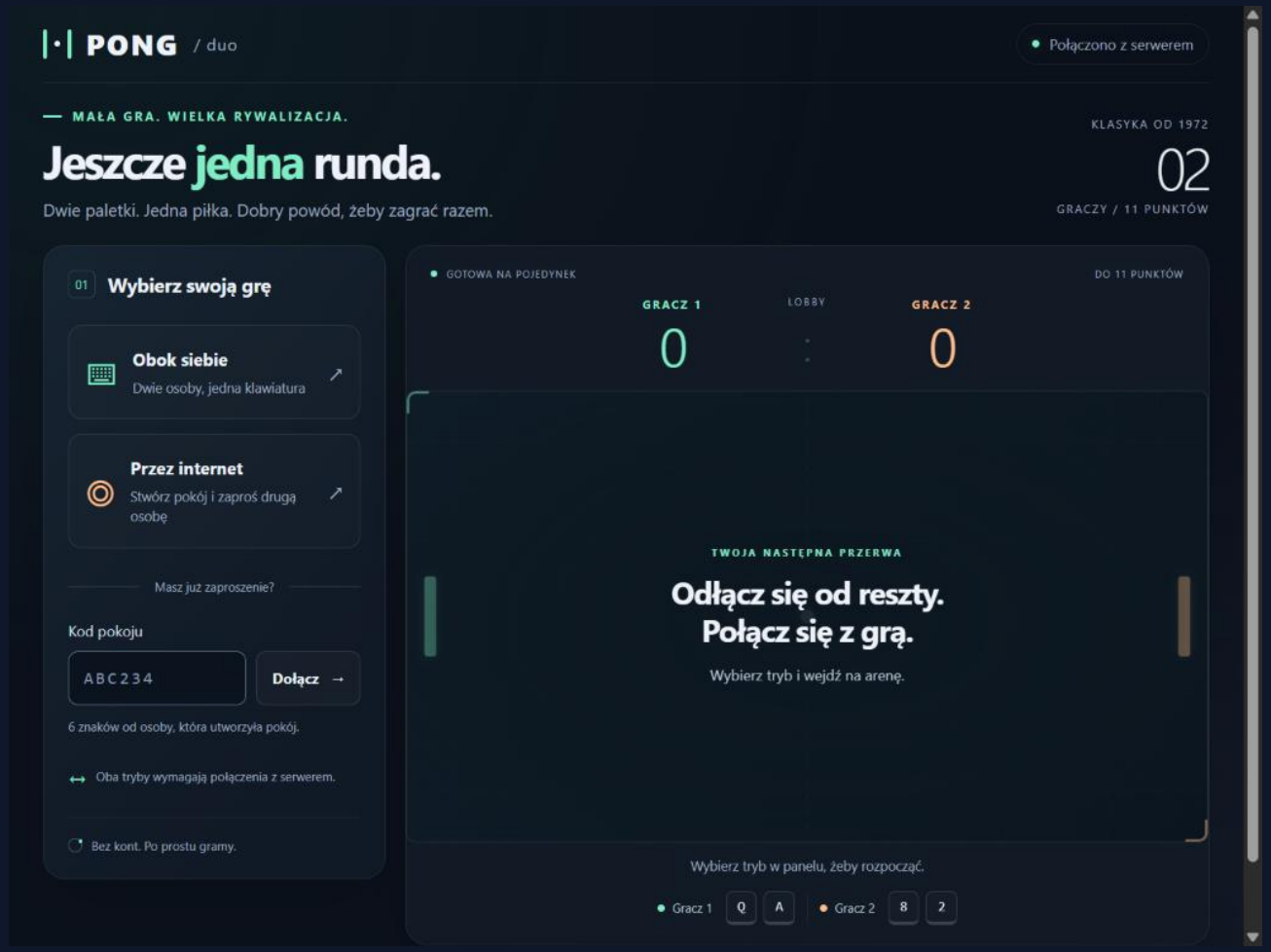
Środowisko

Node 24, firmowy rejestr npm.
Podman w WSL, bez Docker Desktop.
Kontener bez roota, tożsamość o ograniczonych uprawnieniach.

Źródło: .squad/team.md; notatka agenta Backend; commit wdrożeniowy

WYNIK

Pong Arena działa w Azure



ADRES APLIKACJI
ca-fy27-pong.kindocean-fe245546.polandcentral.azurecontainerapps.io

27
testów serwera

Edge
e2e: Playwright

13
socketów w smoke teście

6 + 1
pokoi: online + lokalny

+5 820 linii w trzech commitach

Źródło: commity gałęzi squad/pong-arena; zrzut ekranu działającej aplikacji

WNIOSKI

Dobre praktyki i ograniczenia

DOBRE PRAKTYKI

1. Ustalić zakres i język na starcie — trafiają do team.md i pamięci.
2. Najpierw kontrakt, potem równoległa praca na rozłącznych plikach.
3. Wdrożenie po zielonym build/test, z upoważnieniem do konkretnego zasobu.
4. Testować na prawdziwych gniazdach i w przeglądarce, bez mocków.
5. Commitować .squad/ — wiedza zespołu wędruje z repozytorium.

OGRANICZENIA

1. Jedna replika, stan w pamięci: restart kończy mecze, brak scale-out.
2. Koszt tokenów: wielu agentów i wiele wywołań narzędzi.
3. --yolo wymaga zaufania do repozytorium i środowiska.
4. Przegląd i merge zawsze po stronie człowieka.
5. SDK-first wciąż eksperymentalny.

ŹRÓDŁA

Źródła i linki

Squad (MIT)

github.com/bradygaster/squad

Instalacja

npm @bradygaster/squad-cli · winget · Homebrew

Pong Arena

ca-fy27-pong.kindocean-fe245546.polandcentral.azurecontainerapps.io

Repozytorium studium

tkopacz/fy27-squad-python01 (prywatne), gałąź squad/pong-arena

Pakiet .NET

Squad.Agents.AI 1.0.0 (Microsoft Agent Framework)

Dziękuję — Tomasz Kopacz

Sr Solution Engineer, Microsoft

Przebieg studium zrekonstruowano z artefaktów Squad i commitów.

